

Árboles y grafos

Algoritmo iterativo

Correctitud de un algoritmo iterativo

Invariantes de ciclo

Computación iterativa

Una *computación iterativa* se caracteriza por comenzar en un estado inicial S_0 y transformar ese estado en un conjunto de estados intermedios hasta llegar a un estado final S_j

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_j$$

Todo estado se debe caracterizar por cumplir una condición, llamada *invariante*.

Computación iterativa

¿Qué es una especificación?

*Una **especificación** se define como la descripción de los siguientes parámetros:*

***Entrada:** indica las precondiciones*

***Salida:** indica las poscondiciones*

***Idea iterativa:** muestra cómo deberían cambiar los estados, comenzando desde el inicial hasta llegar al final*

***Estados:** especifica la forma de cada estado en forma de tupla, además, se muestra cuál es el invariante de estado*

***Estado inicial:** muestra los valores que forman el estado inicial*

***Estado final:** muestra los valores que forman el estado final*

***Transformación de estados:** de manera formal especifica cómo se realizan, en términos generales, los cambios de un estado al siguiente*

Computación iterativa

¿Qué es demostrar correctitud de un algoritmo?

Un algoritmo es correcto con respecto a una especificación

Será correcto si para cada entrada que cumple las precondiciones, el algoritmo termina cumpliendo la poscondición

Además, para el caso específico de algoritmos iterativos, se cuenta con un método formal de probar la correctitud

Computación iterativa

Especificación para el cálculo de factorial

Entrada: $\mathcal{N} \geq 0$

Salida: resultado = $\mathcal{N}!$

Idea: Iteración

$(0,1) \rightarrow (1,1) \xrightarrow{\quad} (2,2) \rightarrow (3,6) \rightarrow \dots \rightarrow (\mathcal{N}, \mathcal{N}!)$

Estados: Tupla de la forma (índice, resultado) tal que resultado = índice!
(Invariante)

Estado inicial: índice = 0, resultado = 1

Estado final: índice = $\mathcal{N}$

Transformación de estados:

$(\text{índice}, \text{resultado}) \rightarrow (\text{índice} + 1, \text{resultado} * (\text{índice} + 1))$

Corrección

Un *especificación* es la definición de un problema en términos de su precondition Q y poscondición R

Un algoritmo A es *correcto con respecto a una especificación* si para cada conjunto de valores que cumplen Q , los valores de salida cumplen R

Intencio

Se denota como $\{Q\} A \{R\}$. “ A es correcto con respecto a la precondition Q y a la poscondición R ”

Computación iterativa

Algoritmo para el cálculo de factorial

Factorial(int N){

int indice=0;
int resultado=1;

} Estado inicial

while !(indice==N){

indice=indice +1;

*resultado= resultado * indice;*

}

System.out.println(resultado);

}

Computación iterativa

Algoritmo para el cálculo de factorial

Factorial(int N){

int indice=0;

int resultado=1;

while !(indice==N){

• indice=indice +1;

*resultado= resultado * indice;*

}

System.out.println(resultado);

}

¿Es correcto el algoritmo con respecto a la especificación?

1) $S = (i, r)$ Estado

2) $S_0 = (0, 1)$ Estado inicial

3) Transformación de estados
 $(i, r) \rightarrow (i+1, r(i+1))$

Factorial(int N){

int indice=0;

int resultado=1;

while !(indice==N){

• indice=indice + 1;

resultado= resultado * indice;

}

System.out.println(resultado);

}

4) Invariante

$(i, r) \rightarrow (1, 1)$

$\rightarrow (2, 2) \rightarrow (3, 6)$

$\rightarrow (4, 24)$

$(0, 1) \rightarrow (1, 1 \times 1) \rightarrow$

$(2, 1 \times 1 \times 2) \rightarrow$

$(3, 1 \times 1 \times 2 \times 3)$

$(4, 1 \times 1 \times 2 \times 3 \times 4)$

Invariante $r = i!$

Como se puede observar la INVARIANTE se cumple en todas las iteraciones del ciclo, por lo tanto el programa CORRECTO

Sf $i = N, r = N!$

Demostración de la invariante de ciclo

1. La invariante cumple el estado inicial

$i = 0, r = 1 \quad r = i! \quad 1 = 0!$

2. La invariante cumple el estado final

$i = N \quad r = N! \quad r = i! \quad N! = N!$

3. La invariante cumple transformación de estado

$$(i, r) \rightarrow (i+1, r(i+1))$$

$r=i!$ Inv
Trans

1) $i \rightarrow i+1$

Inv

$$(i+1)!$$

$$1 \times 1 \times 2 \times 3 \times 4 \times \dots \times i \times (i+1)$$

$$(i+1)!$$

Trans

$$r(i+1)$$

$$i!(i+1)$$

$$(1 \times 1 \times 2 \times 3 \times 4 \times \dots \times i)(i+1)$$

El cambio de $i \rightarrow i+1$ conserva la invariante

2) $r \rightarrow r(i+1)$

Inv

$$r(i+1)$$

$$i!(i+1) \rightarrow (i+1)!$$

QED!

4. ¿El algoritmo termina?

$$0, 1, 2, 3, \dots, N$$

Dado que el valor arranca en 0 y $N \geq 0$, en algún momento el índice va alcanzar el valor N, por lo tanto el algoritmo SIEMPRE TERMINA condicionado a que la entrada cumpla la PRECONDICION

Computación iterativa

Especificación para el cálculo de raíz de X

Entrada: $X \geq 0, X \in \mathcal{R}, \delta > 0$

Salida: a tal que $|a^2 - X| \leq \delta$

Idea: Dado X , inicie la aproximación de a con el valor 1.0 y mejorela utilizando el cambio de a por $(a + X/a)/2$

$(1, 1.0) \rightarrow (2, (1.0 + X/1.0)/2) \rightarrow \dots \rightarrow (\mathcal{N}, a)$

Estados: Tupla de la forma (índice, aproximación) tal que $a > 0$ (Invariante)

Estados inicial: $a = 1.0$

Estado final: a tal que $|a^2 - X| \leq \delta$

Transformación de estados: $(\text{índice}, a) \rightarrow (\text{índice} + 1, (a + X/a)/2)$

Computación iterativa

Algoritmo para el cálculo de raíz de X

$\text{raizIterativa}(\text{double } X, \text{double } \text{delta})\{$

$\boxed{\text{double } a=1.0;}$ S_0

$\text{while } (\text{!(Math.abs(a*a-X) <= delta)})\{$

$a = (a + X/a)/2.0;$

$\}$

$\text{System.out.println(a);}$

$\}$

$$S_0 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

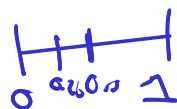
Estimación

$$(i, a) \rightarrow (i+1, (a + \frac{X}{a})/2)$$

$$(0, 1) \rightarrow (1, (1 + \frac{X}{1})/2)$$

$$\rightarrow (2, \left(\frac{1 + \frac{X}{1}}{2} + \frac{X}{\left(\frac{1 + \frac{X}{1}}{2} \right)} \right) / 2)$$

La invariante en este caso es la estimación, que paulatinamente llevar a ser cercano a la raíz, cumpliendo la condición $\text{abs}(a*a-x) < \text{delta}$



64

Computación iterativa

Identifique en los algoritmos *Factorial*(int N) y *raizIterativa*(double X , double δ) los estados inicial y final, así como la transformación dada en la especificación

¿Cómo se manejan las condiciones de entrada en el algoritmo?

Computación iterativa

Algoritmo para el cálculo de factorial

Factorial(int $\mathcal{N}$)

int indice=0;

int resultado=1;



Condiciones iniciales

while !(indice== $\mathcal{N}$)

indice=indice +1;

*resultado= resultado * indice;*

}

System.out.println(resultado);

}

Computación iterativa

Algoritmo para el cálculo de factorial

Factorial(int $\mathcal{N}$)

int indice=0;

int resultado=1;

while !(indice== $\mathcal{N}$)

indice=indice +1;

*resultado= resultado * indice;*

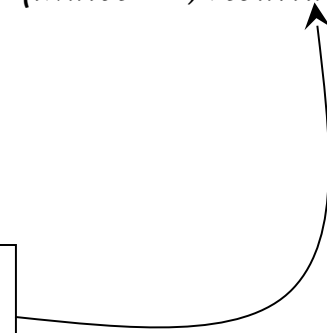
}

System.out.println(resultado);

}

Transformación de estados:

(índice, resultado) $\rightarrow$ (índice +1, resultado(índice+1))*

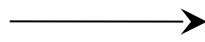


Computación iterativa

Algoritmo para el cálculo de raíz de X

raizIterativa(double X , double δ)

double $a=1.0$;



Condición inicial

*while ($!(\text{Math.abs}(a*a-X) \leq \delta)$)*

$a = (a + X/a)/2.0$;

}

System.out.println(a);

}

Computación iterativa

Algoritmo para el cálculo de raíz de X

raizIterativa(double X , double δ)

double $a=1.0$;

*while (!(Math.abs($a*a-X$)<= δ))*

$a = (a + X/a)/2.0$;

}

System.out.println(a);

}

Transformación de estados:

(índice, a) $\rightarrow$ (índice+1, ($a+X/2$)/2)

Computación iterativa

El esquema de un algoritmo iterativo es el siguiente:

$S \leftarrow S_0$

while ! *isFinal*(*S*) *do*

$S \leftarrow \text{Transform}(S)$

Computación iterativa

Cómo probar que un algoritmo iterativo $\mathcal{A}$ es correcto con respecto a un especificación (precondición Q , poscondición $\mathcal{R}$)

1. *Inicialización*: Pruebe que el estado inicial S_0 cumple el invariante
2. *Invarianza*: Prueba que la transformación conserva el invariante
3. *Éxito*: Si S es un estado final y se cumple el invariante $P \rightarrow \mathcal{R}$
4. *Terminación*: $\mathcal{A}$ termina

Computación iterativa

Computa (int A , int B) {

int $res=0, i=1;$ Estados

while ($i \leq B$) {

$i=i+1;$

$res=res + A;$

}

System.out.println(res);

}

$$i=1 \quad \sum_{p=0}^1 A = 0$$

$$i=2 \quad \sum_{p=0}^2 A = A$$

Qué calcula *Computa*(2,3)?

$$S = (i, res)$$

$$S_0 = (1, 0) \quad \text{Estado inicial}$$

$$(i, res) \rightarrow (i+1, res+A) \quad \text{transformación}$$

$$(1, 0) \rightarrow (2, 0+A), (3, 0+A+A) \\ \rightarrow (4, 0+A+A+A) \rightarrow \dots (B, \text{---})$$

$$\rightarrow (B+1, BA)$$

$$(i, \sum_{p=2}^i A) \quad \text{Invariantr}$$

$$(i-1)A \leftarrow INV$$

$$\sum_{i=1}^n c = cn$$

$$\sum_{p=2}^i A \rightarrow \sum_{p=1}^{i-1} A = (i-1)A$$

$$\sum_{p=2}^i A = \sum_{p=1}^i A - \sum_{p=1}^1 A$$

$$\sum_{p=1}^i A - A = iA - A = (i-1)A$$

Computación iterativa

Computa (*int* $\mathcal{A}$, *int* $\mathcal{B}$) {

int $res=0$, $i=1$;

while ($i \leq \mathcal{B}$) {

$i=i+1$;

$res=res + \mathcal{A}$;

}

System.out.println(res);

}

Q : $\mathcal{A}, \mathcal{B} \in \mathbb{Z}$, $\mathcal{B} > 0$

$\mathcal{R}$: $res = \mathcal{A} * \mathcal{B}$

Computación iterativa

Computa (*int* $\mathcal{A}$, *int* $\mathcal{B}$) {

int $res=0$, $i=1$;

while ($i \leq \mathcal{B}$) {

$i=i+1$;

$res=res + \mathcal{A}$;

}

System.out.println(res);

}

Identifique los estados y su invariante

Computación iterativa

Computa (*int* $\mathcal{A}$, *int* $\mathcal{B}$) {

int $res=0$, $i=1$;

while ($i \leq \mathcal{B}$) {

$i=i+1$;

$res=res + \mathcal{A}$;

}

}

Considere cada estado como el par (i, res)

$(1,0) \rightarrow (2,\mathcal{A}) \rightarrow (3,\mathcal{A}+\mathcal{A}) \rightarrow \dots \rightarrow (\mathcal{B}+1, \mathcal{A} + \dots + \mathcal{A})$

Invariante $\mathcal{P}$: $res = \sum_{i=2}^k A$

Computación iterativa

Probar correctitud

1. *Inicialización*: Pruebe que el estado inicial S_0 cumple el invariante

El estado inicial es $(1,0)$, Se verifica que se cumpla el invariante, se tiene que $i=1$.

Computación iterativa

Probar correctitud

1. *Inicialización*: Pruebe que el estado inicial S_0 cumple el invariante

El estado inicial es $(1,0)$, Se verifica que se cumpla el invariante, se tiene que $i=1$.

$$res = \sum_{i=2}^1 A = 0$$

Computación iterativa

2. Invarianza: Prueba que la transformación conserva el invariante

Se considera que antes de entrar el ciclo, $i=k$ y se prueba.

Si $i=k$,

$$res = \sum_{i=2}^k A = 0$$

Computación iterativa

Computa (*int* $\mathcal{A}$, *int* $\mathcal{B}$) {

int $res=0$, $i=1$;

while ($i \leq \mathcal{B}$) {

$i=i+1$;

$res=res + \mathcal{A}$;

}

System.out.println(res);

}

Computación iterativa

2. Invarianza: Prueba que la transformación conserva el invariante

Se considera que antes de entrar el ciclo, $i=k$ y se prueba.

Si $i=k$

$$res = \sum_{i=2}^k A = 0$$

Al ejecutar la iteración, $i=k+1$:

1. Por cambio, $res = res + A$, reemplazando $res = \sum_{i=2}^k A + A = \sum_{i=2}^{k+1} A$

2. Haciendo $k = k+1$ en la invariante $res = \sum_{i=2}^{k+1} A = 0$

3. Como se ve en ambos obtenemos lo mismos, QUEDA DEMOSTRADO

Computación iterativa

3. *Éxito*: Invariante $\mathcal{P}$ donde S es un estado final $\rightarrow \mathcal{R}$

El ciclo finaliza con $i=B+1$, este es el valor de i en el estado final. Se calcula res.

Computación iterativa

3. *Éxito*: Invariante $\mathcal{P} \wedge S$ es un estado final $\rightarrow \mathcal{R}$

El ciclo finaliza con $i=B+1$, este es el valor de i en el estado final. Se calcula res:

$$res = \sum_{i=2}^{B+1} A = \sum_{i=1}^{B+1} A - A = A * (B+1) - A = A * B$$

Computación iterativa

4. Terminación: A termina

En cada iteración i aumenta, por lo que en algún momento finito tendrá que alcanzar el valor de B y el algoritmo terminará

Computación iterativa

¿Qué calcula $\text{Computa3}(4)$?

Expresa la forma de los estados

Muestre la idea iterativa que presenta el algoritmo

Pruebe la correctitud

Indique la precondition y poscondición

Computa3 (*int* $\mathcal{N}$) {

int $\mathcal{A}$, $\mathcal{B}$, i , j ;

$\mathcal{A}=0$;

$i=1$;

while ($i \leq \mathcal{N}$) {

$\mathcal{B}=1$;

$j=1$;

while ($j \leq 3$) {

$\mathcal{B}=\mathcal{B} * i$;

$j++$;

}

$\mathcal{A}=\mathcal{A}+\mathcal{B}$;

$i++$;

}

System.out.println("Resultado=" + $\mathcal{A}$);

}

Computación iterativa

¿Qué calcula $\text{Opera}(2,3)$?

Expresa la forma de los estados

Muestre la idea iterativa que presenta el algoritmo

Pruebe la correctitud

Indique la precondition y poscondición

Opera (*int* $\mathcal{B}$, *int* $\mathcal{N}$) {

int $\mathcal{A}$, $\mathcal{C}$, $\mathcal{D}$;

$\mathcal{D}=0$;

$\mathcal{A}=1$;

$\mathcal{C}=\mathcal{N}$;

while ($\mathcal{C}\geq 1$) {

$\mathcal{A}=\mathcal{A}*\mathcal{B}$;

$\mathcal{D}=\mathcal{D}+\mathcal{A}$;

$\mathcal{C}--$;

}

System.out.println("Resultado=" + $\mathcal{D}$);

}

Invariante de ciclo: $D = \sum_{i=1}^{N-C} B^i, A = B^{N-C}$

```

BS (int A[], int N){
    int i, j, aux;
    for ( i=1; i < N; i++)
        for ( j=N; j > i ; j){
            if ( A[j] < A[j - 1] ){
                aux=A[j];
                A[j]=A[j-1];
                A[j-1]=aux;
            }
        }
}

```

A partir del procedimiento indicado a continuación, que tiene como entrada un arreglo A indexado de la forma [1..n]. Indicar:

1. Invariante de ciclo para el iterador interno (línea 3)
2. Invariante de ciclo para el iterador externo (línea 2)
3. ¿Que calcula ?

Referencias

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. Introduction to Algorithms, Third Edition (3rd ed.). The MIT Press. Pages 18-20